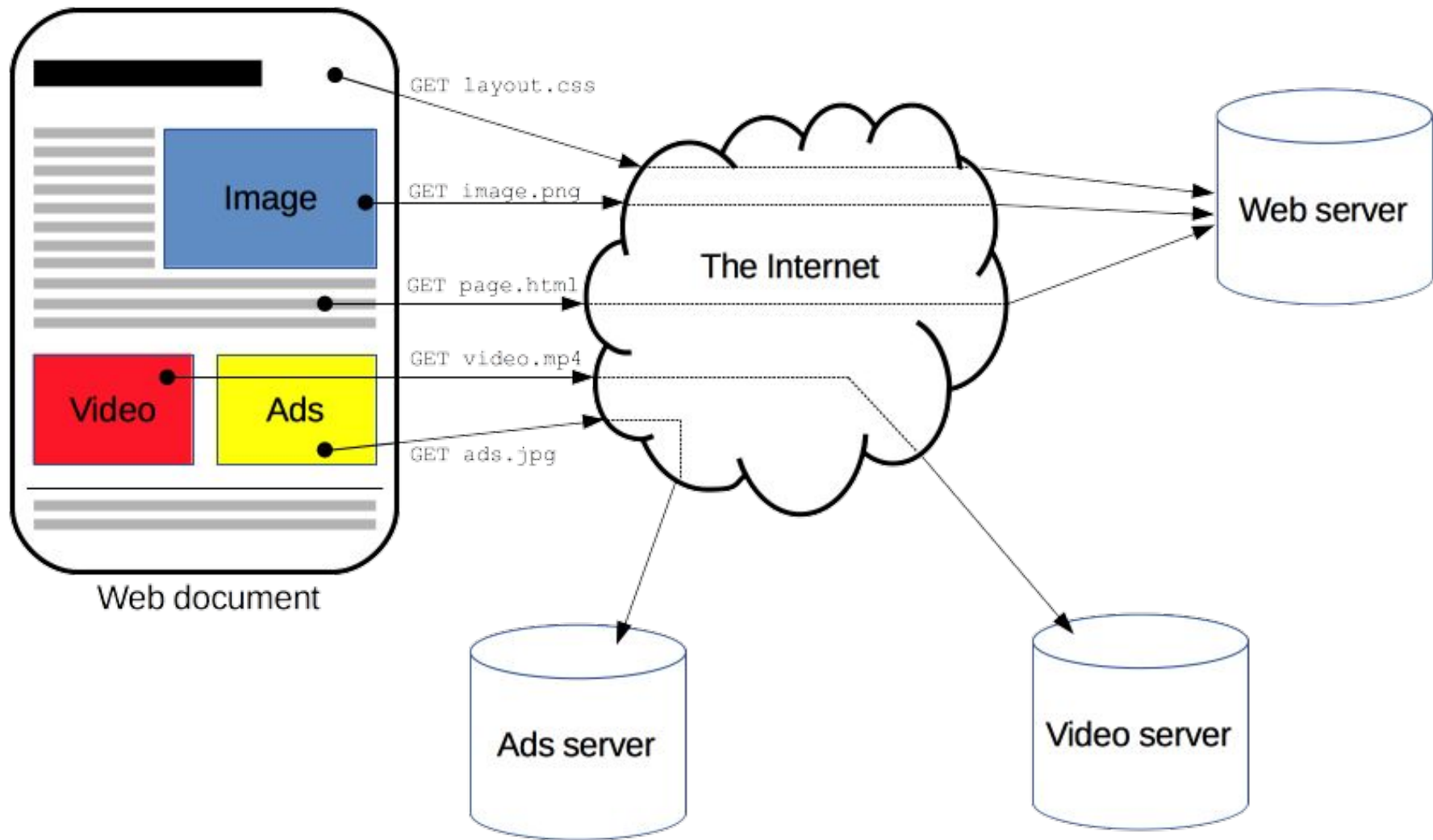


Http & Rest

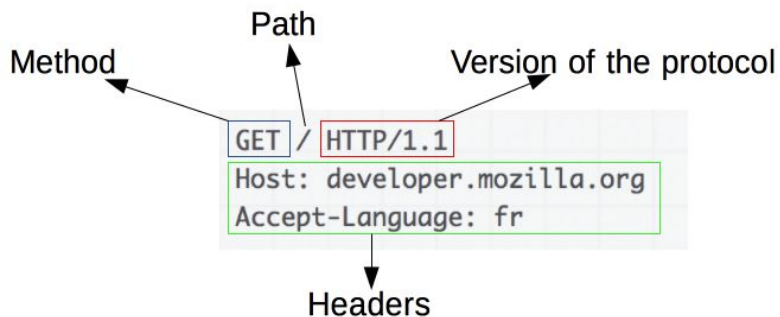
HTTP

- HyperText Transfer Protocol
- Objective is to fetch resources/documents from servers
- Resources:
 - HTML pages
 - Images
 - Videos
 - Ads
 - Text files
 - etc...
 - <https://canaltic.com/blog/?p=6056>

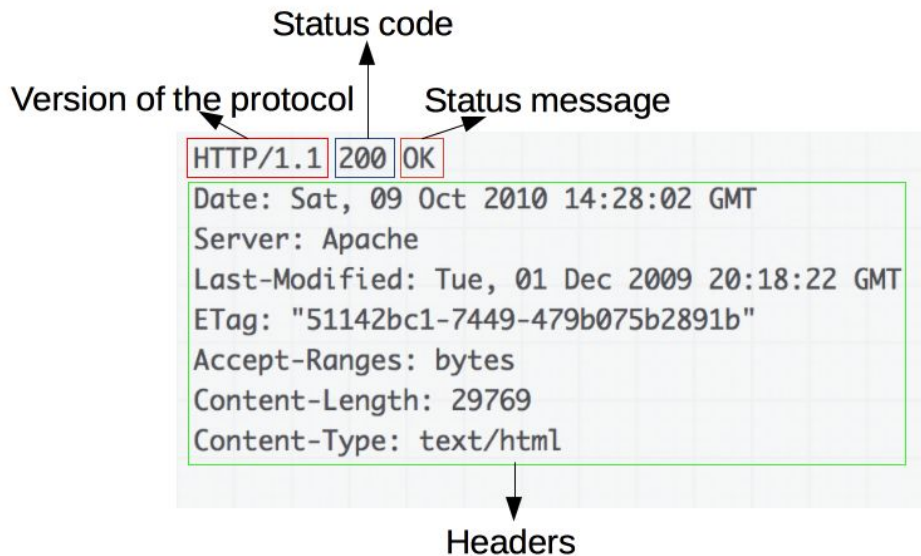


HTTP messages

- Request



- Response



HTTP Headers

- Metadata around the messages
- Additional information
- Some typical usage
 - Authorization: passing token to identify user/session
 - Content-Type: media type of resource
 - User-agent: identify what initiated conversation (browser, etc...)

HTTP verbs (request methods)

- GET
 - retrieves resources
- POST
 - writes a resource
- PUT
 - creates or replace a resource
 - if called several times, same effect
- PATCH
 - modifies a resources or parts of it
- DELETE
 - removes a resource
- HEAD
 - only fetches headers

HTTP response codes

- 200 - range: success codes
 - e.g. 200 OK, 201 created, etc...
- 300 - range: redirections
- 400 - range: client errors (user or browser did something wrong)
 - e.g. 404 resource not found, 409 Conflict, etc...
- 500 - range: server errors
 - something went wrong on server side
 - e.g. 504 gateway timeout, etc...

HTTP Status Codes



1XX
INFORMATIONAL

2XX
SUCCESS

3XX
REDIRECTION

4XX
CLIENT ERROR

5XX
SERVER ERROR

HTTPS

- S for Secure
- HTTP over TLS
- Connections are encrypted
- Must have for sensitive information exchange
 - user information
 - passwords
 - payments
- Mandatory for HTTP version 2

REST

RESTful APIs



{ REST }

REST

- **RE**presentational **S**tate **T**ransfer
- For creating **web services** (RESTful services)
- Used over HTTP
- Use HTTP **verbs**
- Use HTTP **response codes**
- Is **stateless**

REST resources

- Key abstraction in REST
- Represent the entities to manipulate
- Accessing resources
 - <http://mydomain.com/api/v1/resourceName/{id}>
 - <http://mydomain.com/api/v1/resourceName>
- Examples
 - <http://unstravel.netlify.app/api/v1/packages/123456789>
 - <http://unstravel.netlify.app/api/v1/packages>
- Note that
 - resource name is plural

Designing REST APIs - modeling

- Model the resource = entity to represent
- JSON is most common

```
{
  "destinationCode": "CAN",
  "departureCode": "TUC",
  "name": "Cancún, all inclusive",
  "description": "Sleep in the best hotels, all inclusive, access to private beach, etc...",
  "price": 2700,
  "currency": "USD",
  "flights": {
    "departure": [],
    "return": []
  },
  "hotels": [
  ],
  "excursions": [
  ]
}
```

Designing REST APIs - URLs

- Base URL to access resource
 - <http://unstravel.netlify.app/api/v1/packages>
- **GET** <http://unstravel.netlify.app/api/v1/packages> ⇒ search packages
- **GET** <http://unstravel.netlify.app/api/v1/packages/23132> ⇒ retrieve package with id 23132
- **POST** <http://unstravel.netlify.app/api/v1/packages> ⇒ create a package
- **PUT** <http://unstravel.netlify.app/api/v1/packages/23132> ⇒ replace package 23132
- **PATCH** <http://unstravel.netlify.app/api/v1/packages/23132> ⇒ change parts of package 23132

GET - search requests

- <http://unstravel.netlify.app/api/v1/packages> search all
- Use query strings for filtering
 - <http://unstravel.netlify.app/api/v1/packages?destination=CAN&maxPrice=3000>
- Pagination in query strings
 - <http://unstravel.netlify.app/api/v1/packages?limit=5&page=1>

```
{
  "data": [
    {
      "destinationCode": "CAN",
      "departureCode": "TUC",
      "name": "Cancún, all inclusive",
      "description": "Sleep in the best hotels, all inclusive, access to private beach, etc...",
      "price": 2700,
      "currency": "USD"
    },
    {
      "destinationCode": "CAN",
      "departureCode": "TUC",
      "name": "Cancún, breakfast included",
      "description": "Breakfast at the hotel, be free for the rest...",
      "price": 1750,
      "currency": "USD"
    }
  ],
  "page": 3,
  "start": 15,
  "total": 35,
  "next": "/api/v1/packages?start=20&limit=5"
}
```


POST - create resource

- POST to <http://unstravel.netlify.app/api/v1/packages>
 - Creates a package
- Use body contents

PUT - replace a resource

- PUT to <http://unstravel.netlify.app/api/v1/packages/12345>
 - replaces a package contents
- Use body contents
- Provide unique identifier in URL

PATCH - replace some contents

- PATCH to <http://unstravel.netlify.app/api/v1/packages/12345/flights>
 - replaces parts of package contents, here flights
- Use body contents
- Provide unique identifier in URL
- Use URL parameters to identify parts to replace

DELETE - removes a resource

- DELETE to <http://unstravel.netlify.app/api/v1/packages/12345>
 - removes corresponding package
- Body not needed
- Provide unique identifier in URL

Versioning considerations

- Version APIs to identify compatibility
- Breaking changes in contract needs major version change
- Design for
 - backward compatibility
 - forward compatibility

RESTful API: 5 CLAVES DE DISEÑO EFICIENTE Y ESCALABLE

DISEÑO DE URI

Diseña tus endpoints con **nombres de recursos** (sustantivos), no acciones (verbos).
Ej.: **/users** en lugar de **/getUsers**

EVITAR URL MUY ANIDADAS

Mantener las rutas lo más simples y claras posible.
Ej.: **/authors/12?category=2** es preferible a **/authors/12/categories/2**



USO CORRECTO DE MÉTODOS HTTP

Asocia cada operación con el método HTTP adecuado: **GET** (obtener), **POST** (crear), **PUT/PATCH** (actualizar), **DELETE** (eliminar).

RESPUESTAS EN JSON ESTRUCTURADO

Estandariza no solo los encabezados (Content-Type: application/json, Accept: application/json), sino también el formato del cuerpo de la respuesta.

CÓDIGOS DE ESTADO

Responde con códigos precisos según la operación realizada:
1xx: Informational, **2xx**: Success, **3xx**: Redirection, **4xx**: Client Errors, **5xx**: Server Errors

Practice

Ejercicio

- En cada entidad (CLIENTES, TRANSPORTADORES)
- Definir el resource principal
- Definir cuales endpoints deberian existir para manipular el resource principal
 - Creacion de un resource
 - Búsqueda de los resources (y cuales filtros)
 - Obtener detalles de 1 resource
 - Cambiar un resource
 - Remover un resource
- Tener en cuenta
 - completitud de la información
 - extensibilidad de la API
 - Problemas con paginación